FINAL CAPSTONE REPORT | AI CAPSTONE (MODULE 27)

ResuMatch AI

Intelligent Resume-JD Gap Analysis & Career Coaching System

Student / Mentor	Md. Shafat Hossain / Sazzad Hossain
Date	October 2026
Live app / API	https://resumatch-ai-five.vercel.app/ / https://resumatch-ai-rizc.onrender.com/docs
Repository	https://github.com/mdshafathossain-tech/resumatch-ai

Abstract. ResuMatch AI helps job seekers see how well a resume fits a specific job description (JD) and how to improve it. A user uploads a PDF resume and pastes a JD; the system returns overall, hard-skill and soft-skill match scores, missing keywords, skill gaps and AI-rewritten bullet points, and stores each analysis for trend tracking. It combines a Next.js 14 frontend, a FastAPI backend, a Supabase PostgreSQL database secured with Row Level Security (RLS), and Google Gemini analysis through the google-genai SDK with a three-model fallback chain.

1 INTRODUCTION AND BACKGROUND

- **Opaque rejection.** ATS pipelines lean on keyword matching, so a strong candidate can be filtered out for wording differences ("dashboarding" vs. "data visualization") and never learn why.
- **Unknown fit.** Candidates reuse one resume for many roles and cannot see which required skills it under-represents.
- **Generic advice.** Free tools give checklists; role-specific bullet rewriting is usually paywalled.

Objectives (all delivered): calibrated overall, hard and soft scores; missing keywords and prioritized gaps; role-aligned rewrites without fabricated facts and with [X%] metric placeholders; persistent history with a score-trend dashboard; reliability under free-tier AI quotas; public deployment with documentation.

Scope: text-based PDF resumes up to 5 MB and English JDs of 50-10,000 characters; authentication, OCR and multilingual support are out of scope.

2 SYSTEM ARCHITECTURE AND DESIGN DECISIONS

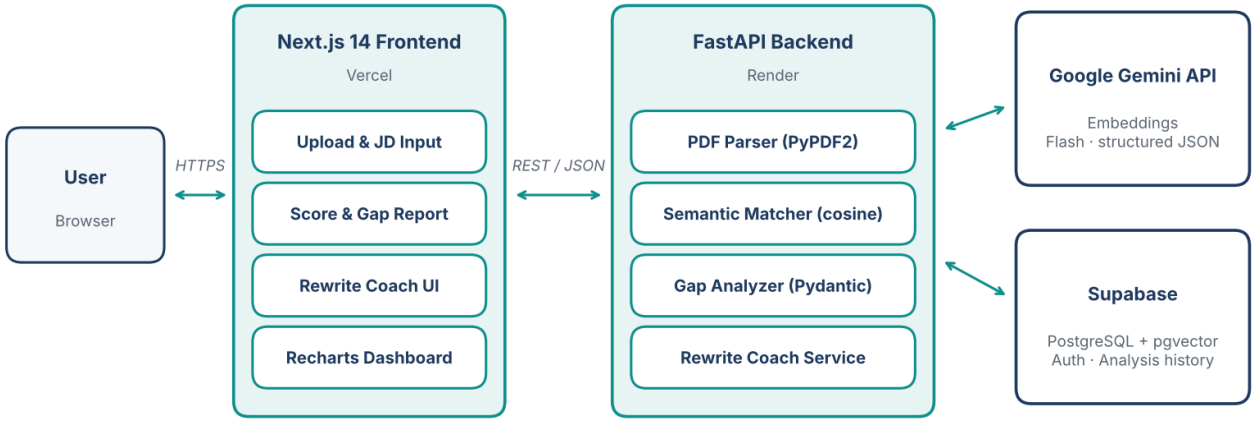


Figure 1. System architecture

Layer	Technology	Responsibility
Frontend	Next.js 14 (App Router), TypeScript, Tailwind, Lucide, Recharts, Framer Motion	Upload form, loading UX, report rendering, history dashboard
Backend	FastAPI, Uvicorn, Pydantic v2, PyPDF2	Validation, PDF extraction, AI orchestration, persistence, REST API
AI / Data	Google Gemini (google-genai); Supabase PostgreSQL (JSONB, RLS)	Structured analysis; durable history
Hosting	Vercel (frontend), Render (backend)	UI delivery and API hosting

Backend. Layered into routers (HTTP and error mapping), services (PDF and AI logic), schemas (Pydantic contracts), and config/database. Configuration fails fast at startup; blocking PDF, Gemini and Supabase calls run in a thread pool so the event loop stays responsive; every failure maps to a specific HTTP status with a safe message. **Frontend.** Two routes (analyze and history) share one report component; a slide-over viewer shows past reports. The UI is mobile-first and accessible (keyboard dropzone, ARIA labels, reduced-motion support), mirrors server validation for instant feedback, and uses a 120-second timeout, staged progress UI and request cancellation.

Decision	Alternative	Rationale
Single structured LLM call for scoring	Embedding cosine similarity; multi-step chains	One call yields scores, gaps and rewrites with consistent reasoning, lower latency and quota use; embeddings alone cannot explain a low score or write rewrites.
RLS plus server-held service_role key	Client-side database access	The browser never touches the database; the attack surface is the validated API.
JSONB for list fields	Normalized child tables	Lists are read and written as a unit; simpler schema.
Do not store raw resume text	Store full text	Data minimization; only the analysis and JD are needed for history.

Design change from the proposal. The first proposal used embedding cosine similarity for the match score. Implementation consolidated scoring into a rubric-driven LLM analysis because it gives explainable sub-scores and rewrites in one call. The pgvector extension remains enabled so a hybrid embedding signal can be added later.

3 AI IMPLEMENTATION AND WORKFLOW RATIONALE

1. **Validate and extract:** check extension, content type, size and PDF signature; extract text with PyPDF2; normalize it (Unicode NFKC, de-hyphenation); reject scanned or unreadable files.
2. **Build the prompt:** wrap resume and JD in <resume> and <job_description> tags after stripping user-supplied copies; cap at 15,000 and 8,000 characters.
3. **Call Gemini** with a system instruction, temperature 0.2 and JSON response mode.
4. **Parse defensively:** strip code fences, salvage the outermost JSON object, normalize values, validate with Pydantic.
5. **Persist and respond:** store the result in Supabase and return the stored record.

Technique	Rationale
System instruction with explicit rubric	Consistent, auditable scoring.
XML tag isolation and "treat as data" rule	Reduces prompt-injection risk from untrusted resume or JD text.
Verbatim original bullets	Grounds rewrites in real content and prevents invented experience.
[X%] placeholders	Prompts users to add real, verifiable metrics instead of fabricated ones.
JSON mode plus fixed schema	Predictable data for the UI.

Multi-model fallback. Attempts rotate through gemini-3.8-flash, gemini-3.6-flash and gemini-3.5-flash-lite (then the primary again), up to four attempts. Rotation matters because overload and quota limits are often per model.

Provider outcome	Action
429 rate limit	Next model after 2.0 s x attempt plus jitter
500 / 503 / 504	Next model after 1.5 s x attempt plus jitter
403 / 404 (model unavailable)	Next model immediately
Invalid JSON or schema failure	Retry after about 1 s
Other 4xx	Fail fast with a safe message

4 KEY RESULTS, CALIBRATION AND SCORING METHODOLOGY

Score	Definition
Hard skills	Coverage of technical skills, tools, technologies, certifications and domain knowledge the JD requires.
Soft skills	Evidence of communication, leadership, teamwork, problem-solving and other soft skills the JD asks for.
Overall match	Weighted blend: about **60% hard skills, 25% experience and role relevance, 15% soft skills**.
Bands (UI colors)	75-100 Strong match (green); 50-74 Partial match (yellow); 0-49 Weak match (red).

- **Calibration:** the prompt demands realistic scores (90+ only for near-perfect matches, below 40 for weak ones); temperature 0.2 limits variance; values are coerced to integers and clamped to 0-100; lists are capped (15 keywords, 10 gaps, 5 rewrites).
- **Functional results:** end-to-end flow (upload, analysis, persistence, response) verified with HTTP 200; history list, detail view and trend chart verified; invalid input returns the documented 4xx errors; observed provider 503s were absorbed by the retry and fallback layer; resilience logic verified with a simulated provider (503, 404, repeated 429, fenced JSON, 400).
- **Limitation:** scores are model-judged estimates, not statistically validated measurements. Measured results from the deployed system:

Test	Method	Result	Target	Status
Matched vs. mismatched pairs	[3] matching and [1] non-matching resume-JD pairs	Matched avg: [__] vs. Mismatched avg: [__]	Matched ≥ 70, mismatched ≤ 40, gap ≥ 30	Data pending
Repeatability	Same pair run [4] times	Range 22 - 38 (runs: 22, 28, 38; mean 29.3; spread 16 points)	Spread ≤ 5 points	Not met
Latency	[4] end-to-end analyses	Median 4.65 s, max 6.2 s (n = 4: 4.0, 4.2, 5.1, 6.2)	10s - 20s typical	Met
Schema validity	First-attempt valid responses out of [N]	[__] % ([__] of [__] requests)	≥ 95% after retries	Data pending

5 SECURITY, DATA PERSISTENCE AND PERFORMANCE

Area	Control
Uploads	Extension, content-type, 5 MB size and %PDF- signature checks; empty, encrypted or unreadable PDFs rejected.
Prompt injection	Role separation, XML boundary tags, tag stripping, strict output validation, length caps; the model has no tools or network access.
Secrets	service_role and Gemini keys only in backend environment variables; .env files git-ignored.
Database	RLS on all tables with no anon or authenticated policies (deny by default) plus REVOKE ALL for public roles; only the backend reads or writes.
Input and errors	UUID-typed path parameters, bounded pagination, parameterized queries; provider errors and stack traces stay in server logs.
Privacy	Raw resume text is never stored.

Persistence. The analysis_history table stores the filename, title, JD, three scores and three JSONB arrays, with CHECK constraints (0-100), non-null array defaults, and indexes on created_at (descending) and user_id for history queries and future per-user filtering. **Performance.** Thread-pool offloading keeps the API responsive; history queries are paginated and select only needed columns; a cached Supabase client avoids reconnects; input caps and a single LLM call bound latency and quota; the frontend uses static generation, route-level code splitting, request cancellation and a progress UI.

6 CHALLENGES OVERCOME AND LESSONS LEARNED

Challenge	Resolution	Lesson
Gemini 503/429 under load	Retries with backoff and three-model rotation; distinct log lines and user messages	Treat the AI provider as unreliable; design for degraded modes.
Model names change or become restricted	Models and fallbacks configurable through environment variables	Never hard-wire model identifiers.

Challenge	Resolution	Lesson
SDK migration to google-genai	Rewrote client, error handling (APIError.code) and config objects	Track SDK deprecations early.
Output not always clean JSON	Prompt, JSON mode, repair parser, Pydantic validation	Validate model output at the boundary.
Blocking SDK in async framework	run_in_threadpool for blocking calls	Async frameworks need care with sync libraries.
Securing the database	RLS deny-by-default with the service_role key on the backend	A leaked public key must not equal data access.
Scanned PDFs, CORS wildcard, clipboard	Minimum-text check; credentials off with * origin; legacy copy fallback	Fail early with guidance; harden dev defaults.

7 REAL-WORLD IMPACT, NOVELTY AND FUTURE ENHANCEMENTS

- **Impact:** free ATS-style feedback and role-specific coaching, valuable for fresh graduates and career switchers; every output maps to a concrete edit; the history dashboard shows whether each revision improved the match.
- **Novelty:** metric placeholders ([X%]) mark exactly where a real number would add impact instead of fabricating achievements; explainable hard/soft/overall decomposition; production-grade AI resilience (model rotation, jittered backoff, repair parser) in a student project.
- **Limitations:** no authentication yet (history is shared per deployment, so avoid sensitive resumes on the public instance); scores not validated on a labeled dataset; no inbound rate limiting; text-based PDFs and English only; CORS open by default.
- **Future work (priority order):** Supabase Auth with per-user history and owner-only RLS; inbound rate limiting and CORS restriction; labeled evaluation set; hybrid scoring with pgvector embeddings; caching and native response schemas; resume export; OCR and multilingual support.

8 CONCLUSION

ResuMatch AI delivers a complete, deployed AI product: a responsive Next.js interface, a validated FastAPI service, a secured Supabase database, and a resilient Gemini analysis layer. Its main contributions are an explainable score decomposition, honest rewrite coaching with metric placeholders, and engineering practices that keep an LLM-backed service dependable on free-tier infrastructure. The remaining work (authentication, quantitative validation, hybrid scoring) is clearly scoped and builds directly on the current architecture.

References: Google Gemini API docs (ai.google.dev/gemini-api/docs); FastAPI (fastapi.tiangolo.com); Next.js (nextjs.org/docs); Supabase Row Level Security (supabase.com/docs/guides/database/postgres/row-level-security); Recharts (recharts.org).